King Saud University

CSC212 – Data Structures

Ride-Sharing System

Project Phase 1

Worst-Case Time Complexity (Big-O) Analysis Report

Team members

Member	Name	Student ID	Responsibility
Person 1	Abdullah Abonadi	446103718	Riders & Drivers; shared LinkedList
Person 2	Turki Alshahrani	446104322	Rides & Scheduling Structures
Person 3	Abdulaziz _____	446100871	Main System & Integration

October 31, 2026

Division of work

The project was split by class. Each member implemented, tested and analysed the classes below; the analyses are combined in this report in the same order.

Person 1 – Riders & Drivers

Files: Person.java (abstract), Rider.java, Driver.java, RiderList.java, DriverList.java, and the shared LinkedList.java / Node.java.

- Rider and driver fields, constructors, getters and setters; phone-number and vehicle-plate validation.
- Linked lists of riders and drivers kept sorted by ID; searching, adding and removing records.
- Rejecting duplicate IDs, emails and vehicle plates; removal by name, home city or vehicle type removes every matching record.

Person 2 – Rides & Scheduling Structures

Files: DateTime.java, Ride.java (abstract), PrivateRide.java, SharedRide.java, RideList.java.

- Chronological DateTime comparison; ride attributes, constructors, getters and setters.
- Private rides with one rider; shared rides with multiple participants (add, remove and search participants).
- RideList kept sorted alphabetically by pickup location; ride searching and removal.

Person 3 – Main System & Integration

Files: RideSharingSystem.java, Main.java, CSV loading, final integration and submission.

- All IRideSharingSystem methods; rider and driver management through RiderList and DriverList.
- Private and shared ride scheduling with conflict checking for riders and drivers; cascade deletion when a rider or driver is removed.
- Loading riders_100.csv, drivers_30.csv and rides_40.csv; the 12-option command-line menu; integration and the final ZIP and PDF.

Shared responsibilities

- A custom LinkedList<T> is used throughout; no java.util collection classes are used and the provided interfaces are unmodified.
- Each member tested their own classes before integration and wrote the worst-case Big-O analysis, with calculation steps, for their own methods.

Contents

Section	Author	Classes covered	Pages
Person 1 – Worst-Case Time Complexity Analysis	Person 1	LinkedList, Person, Rider, Driver, RiderList, DriverList	3–9
Person 2 – Worst-Case Time Complexity Analysis	Person 2	DateTime, Ride, PrivateRide, SharedRide, RideList	10–17
Time Complexity Analysis: class RideSharingSystem	Person 3	RideSharingSystem	18–28
Time Complexity Analysis: class Main	Person 3	Main	29–34

Page numbers refer to the numbers printed at the bottom centre of each page of this combined report. n denotes the size of the list each method works on; Person 2 also uses k for the number of riders on one shared ride.

Person 1 – Worst-Case Time Complexity Analysis

Person, Rider, Driver, RiderList, DriverList and the shared LinkedList<T>

Counting rules

Every Person 1 method runs in O(1), O(n) or O(n^2). Each table gives the steps per execution (s/e), the frequency and the total for the worst case, in the same style as the Person 2 section.

- **n** = number of riders stored in RiderList, number of drivers stored in DriverList, or number of nodes in a LinkedList.
- Each simple statement or condition counts as 1 step.
- A call to a constant-time method counts as 1 step: getters, `equals`, `compareTo`, and the LinkedList methods `empty`, `findFirst`, `findNext`, `retrieve`, `update`, `insert` and `last`.
- The phone and plate format checks (`matches`) read a short string of fixed length (10 digits, or 3 letters + 4 digits), so each counts as 1 step; their cost does not depend on n.
- `LinkedList.remove()` walks from the head to find the node before the cursor, so it counts as 2n + 2 in the worst case (derived below).
- A call to a method analysed in this section counts as its own worst-case total.

LinkedList<T> (shared structure)

The course cursor-based singly linked list. It keeps a head pointer and a cursor (current). Every operation is O(1) except remove, which has to find the predecessor of the cursor because the nodes have no backward link.

insert (worst case: list not empty, so the new node is linked after the cursor)

Statement	s/e	Freq	Total
`Node<T> node = new Node<>(e);`	1	1	1
`if (empty())`	1	1	1
`node.next = current.next;`	1	1	1
`current.next = node;`	1	1	1
`current = node;`	1	1	1
Total			**5 → O(1)**

remove (worst case: the cursor is on the last of n nodes, so the scan for the predecessor is longest)

Statement	s/e	Freq	Total
`if (current == head)`	1	1	1
`Node<T> previous = head;`	1	1	1
`while (previous.next != current)`	1	n−1	n−1
`previous = previous.next;`	1	n−2	n−2
`previous.next = current.next;`	1	1	1
`if (current.next == null)`	1	1	1
`current = head;`	1	1	1
Total			**2n + 2 → O(n)**

When the cursor is on the head the first branch runs instead and the call costs 3 steps. Person 2's SharedRide tables count this same call as k + 3; the two counts differ only in the constant and both are linear in the list length, so the orders agree.

Constant-time methods

Method	Steps (worst case)	Total	Big-O
`LinkedList() (constructor)`	head = current = null	1	O(1)
`findFirst, findNext, retrieve, update`	one assignment or one return	1	O(1)
`full, empty, last`	one comparison	1	O(1)

Person, Rider and Driver

These three classes only store and validate fields. None of their methods contains a loop or touches a list of size n, so every method runs a fixed number of steps and is O(1). The rideHistory list is created empty in the Person constructor and is only read through getRideHistory; the system classes fill it.

Class	Method	Steps (worst case)	Total	Big-O
Person	`constructor`	id 1 + name 1 + setPhoneNumber 2 + new LinkedList 1	5	O(1)
Person	`setPhoneNumber`	null/format check 1 + assignment 1	2	O(1)
Person	`getId, getName, setName, getPhoneNumber, getRideHistory`	one return or one assignment	1	O(1)
Person	`toString`	one concatenated return	1	O(1)
Rider	`constructor`	Person constructor 5 + email 1 + homeCity 1	7	O(1)
Rider	`getEmail, setEmail, getHomeCity, setHomeCity`	one return or one assignment	1	O(1)
Rider	`compareTo`	null check 1 + Integer.compare 1	2	O(1)
Rider	`toString`	one concatenated return	1	O(1)
Driver	`constructor`	Person constructor 5 + setVehiclePlate 2 + vehicleType 1	8	O(1)
Driver	`setVehiclePlate`	null/format check 1 + assignment 1	2	O(1)
Driver	`getVehiclePlate, getVehicleType, setVehicleType`	one return or one assignment	1	O(1)
Driver	`compareTo`	null check 1 + Integer.compare 1	2	O(1)
Driver	`toString`	one concatenated return	1	O(1)

RiderList

RiderList keeps the riders in a LinkedList<IRider> sorted by ID and a counter for size. Every search walks the list from its first node with the cursor, so the searches are O(n). Removing one rider is O(n). Removing every rider that matches a name or a home city is $O(n^2)$, because each removal makes LinkedList.remove scan for the predecessor again.

findById (worst case: the ID is larger than every stored ID, so the whole list is scanned and null is returned at the last node)

Statement	s/e	Freq	Total
`if (list.empty()) return null;`	1	1	1
`list.findFirst();`	1	1	1
`while (true)`	1	n	n
`IRider value = list.retrieve();`	1	n	n
`if (value.getId() == id) return value;`	1	n	n
`if (value.getId() > id || list.last()) return null;`	1	n	n
`list.findNext();`	1	n−1	n−1
Total			**5n + 2 → O(n)**

Because the list is sorted, the scan stops early as soon as it passes an ID larger than the one searched for, but the worst case still visits all n riders.

findByEmail (worst case: no rider has the email, so the whole list is scanned)

Statement	s/e	Freq	Total
`if (key == null || list.empty()) return null;`	1	1	1
`list.findFirst();`	1	1	1
`while (true)`	1	n	n
`if (key.equals(list.retrieve().getEmail())) return list.retrieve();`	1	n	n
`if (list.last()) return null;`	1	n	n
`list.findNext();`	1	n−1	n−1
Total			**4n + 2 → O(n)**

findByName (worst case: every rider has the name, so every rider is copied into the result)

Statement	s/e	Freq	Total
`LinkedList<IRider> result = new LinkedList<>();`	1	1	1
`if (key == null || list.empty()) return result;`	1	1	1
`list.findFirst();`	1	1	1
`while (true)`	1	n	n
`if (key.equals(list.retrieve().getName()))`	1	n	n
`result.insert(list.retrieve());`	1	n	n
`if (list.last()) return result;`	1	n	n
`list.findNext();`	1	n−1	n−1
Total			**5n + 2 → O(n)**

result.insert is O(1) because the result list inserts after its own cursor, so the matches are appended in ID order without any scan.

findByHomeCity (worst case: every rider lives in the city)

Statement	s/e	Freq	Total
`LinkedList<IRider> result = new LinkedList<>();`	1	1	1
`if (key == null || list.empty()) return result;`	1	1	1
`list.findFirst();`	1	1	1
`while (true)`	1	n	n
`if (key.equals(list.retrieve().getHomeCity()))`	1	n	n
`result.insert(list.retrieve());`	1	n	n
`if (list.last()) return result;`	1	n	n
`list.findNext();`	1	n−1	n−1
Total			**5n + 2 → O(n)**

getAll (worst case: n riders are copied)

Statement	s/e	Freq	Total
`LinkedList<IRider> result = new LinkedList<>();`	1	1	1
`if (list.empty()) return result;`	1	1	1
`list.findFirst();`	1	1	1
`while (true)`	1	n	n
`result.insert(list.retrieve());`	1	n	n
`if (list.last()) return result;`	1	n	n
`list.findNext();`	1	n−1	n−1
Total			**4n + 2 → O(n)**

add (worst case: the new ID is larger than every stored ID, so both uniqueness checks scan the whole list and the rider is inserted last)

Statement	s/e	Freq	Total
`if (rider == null) return false;`	1	1	1
`if (findById(rider.getId()) != null || findByEmail(rider.getEmail()) != null) return false;`	(5n+2) + (4n+2) + 1	1	9n + 5
`if (list.empty())`	1	1	1
`list.findFirst();`	1	1	1
`while (list.retrieve().getId() < rider.getId() && !list.last())`	1	n	n
`list.findNext();`	1	n−1	n−1
`if (list.retrieve().getId() > rider.getId())`	1	1	1
`list.insert(rider);`	1	1	1
`count++;`	1	1	1
`return true;`	1	1	1
Total			**11n + 11 → O(n)**

If the new rider belongs before the node the cursor stopped on, the list has no insert-before operation, so the code overwrites that node with the new rider (update) and re-inserts the displaced rider after it (insert). That branch costs 3 steps instead of 1 and does not change the order of growth. A duplicate ID or email is rejected by the second statement.

removeById (worst case: the rider is the last node, so findById scans all n riders and remove walks to the predecessor)

Statement	s/e	Freq	Total
`if (findById(id) == null) return false;`	(5n+2) + 1	1	5n + 3
`list.remove();`	2n + 2	1	2n + 2
`count--;`	1	1	1
`return true;`	1	1	1
Total			**7n + 7 → O(n)**

removeByEmail (worst case: the rider is the last node)

Statement	s/e	Freq	Total
`if (findByEmail(key) == null) return false;`	(4n+2) + 1	1	4n + 3
`list.remove();`	2n + 2	1	2n + 2
`count--;`	1	1	1
`return true;`	1	1	1
Total			**6n + 7 → O(n)**

removeByName (worst case: many riders match and they sit after the riders that do not)

Statement	s/e	Freq	Total
`if (key == null || list.empty()) return 0;`	1	1	1
`int removed = 0;`	1	1	1
`list.findFirst();`	1	1	1
`while (true)`	1	n	n
`boolean wasLast = list.last();`	1	n	n
`if (key.equals(list.retrieve().getName()))`	1	n	n
`list.remove();`	≤ 2n + 2	≤ n	≤ $2n^2 + 2n$
`count--;`	1	≤ n	≤ n
`removed++;`	1	≤ n	≤ n
`else if (!wasLast) list.findNext();`	1	≤ n	≤ n
`if (wasLast) return removed;`	1	n	n
Total			**$2n^2 + 9n + 3$ → $O(n^2)$**

The loop visits each of the n nodes exactly once: a node is either removed (the cursor moves on to its successor) or skipped with findNext. The quadratic term comes from LinkedList.remove, which has no predecessor pointer and rescans from the head every time. The bound is tight in its order of growth: with n/2 non-matching riders followed by n/2 matching ones, every one of the n/2 removals walks past the n/2 untouched nodes, about $n^2/4$ steps in total. (When every rider matches, each removal is of the head and costs 3 steps, so that input is O(n), not the worst case.)

removeByHomeCity (worst case: many riders match and they sit after the riders that do not)

Statement	s/e	Freq	Total
`if (key == null || list.empty()) return 0;`	1	1	1
`int removed = 0;`	1	1	1
`list.findFirst();`	1	1	1
`while (true)`	1	n	n
`boolean wasLast = list.last();`	1	n	n
`if (key.equals(list.retrieve().getHomeCity()))`	1	n	n
`list.remove();`	≤ 2n + 2	≤ n	≤ $2n^2 + 2n$
`count--;`	1	≤ n	≤ n
`removed++;`	1	≤ n	≤ n
`else if (!wasLast) list.findNext();`	1	≤ n	≤ n
`if (wasLast) return removed;`	1	n	n
Total			**$2n^2 + 9n + 3 \rightarrow O(n^2)$**

Same loop as removeByName with the home city as the key; the same worst-case input gives the same count.

size

Returns the maintained counter: 1 step, O(1).

DriverList

DriverList is the same structure as RiderList with drivers instead of riders: a LinkedList<IDriver> kept sorted by ID plus a size counter. Each method is statement-for-statement the same as the RiderList method listed beside it, with the vehicle plate taking the place of the email as the second unique key and the vehicle type taking the place of the home city as the non-unique key. The comparison of two VehicleType values is one step, like a string equals. The worst cases and step counts are therefore identical.

DriverList method	Same statements as RiderList	Worst case	Total (worst case)	Big-O
`add`	`add`	new ID larger than every stored ID; plate check scans all n	11n + 11	O(n)
`findById`	`findById`	ID larger than every stored ID	5n + 2	O(n)
`findByVehiclePlate`	`findByEmail`	plate not present	4n + 2	O(n)
`findByName`	`findByName`	every driver matches	5n + 2	O(n)
`findByVehicleType`	`findByHomeCity`	every driver matches	5n + 2	O(n)
`getAll`	`getAll`	n drivers copied	4n + 2	O(n)
`removeById`	`removeById`	driver is the last node	7n + 7	O(n)
`removeByVehiclePlate`	`removeByEmail`	driver is the last node	6n + 7	O(n)
`removeByName`	`removeByName`	matching drivers after non-matching ones	$2n^2 + 9n + 3$	$O(n^2)$
`removeByVehicleType`	`removeByHomeCity`	matching drivers after non-matching ones	$2n^2 + 9n + 3$	$O(n^2)$

DriverList method	Same statements as RiderList	Worst case	Total (worst case)	Big-O
`size`	`size`	—	1	O(1)

Duplicate handling: add rejects a driver whose ID or vehicle plate already exists (both checks are O(n)), which is how the uniqueness of plates required by IDriver is enforced.

Final summary

Worst-case results for all Person 1 methods. n is the number of riders, drivers or nodes in the list concerned.

Class	Method	Total (worst case)	Big-O
LinkedList	`insert`	5	O(1)
LinkedList	`remove`	2n + 2	O(n)
LinkedList	`all other methods`	1	O(1)
Person	`all methods`	constant (≤ 5)	O(1)
Rider	`all methods`	constant (≤ 7)	O(1)
Driver	`all methods`	constant (≤ 8)	O(1)
RiderList	`add`	11n + 11	O(n)
RiderList	`findById`	5n + 2	O(n)
RiderList	`findByEmail`	4n + 2	O(n)
RiderList	`findByName, findByHomeCity`	5n + 2	O(n)
RiderList	`getAll`	4n + 2	O(n)
RiderList	`removeById`	7n + 7	O(n)
RiderList	`removeByEmail`	6n + 7	O(n)
RiderList	`removeByName, removeByHomeCity`	$2n^2 + 9n + 3$	$O(n^2)$
RiderList	`size`	1	O(1)
DriverList	`add`	11n + 11	O(n)
DriverList	`findById`	5n + 2	O(n)
DriverList	`findByVehiclePlate`	4n + 2	O(n)
DriverList	`findByName, findByVehicleType`	5n + 2	O(n)
DriverList	`getAll`	4n + 2	O(n)
DriverList	`removeById`	7n + 7	O(n)
DriverList	`removeByVehiclePlate`	6n + 7	O(n)
DriverList	`removeByName, removeByVehicleType`	$2n^2 + 9n + 3$	$O(n^2)$
DriverList	`size`	1	O(1)

Person 2 – Worst-Case Time Complexity Analysis (CSC212 Ride-Sharing)

Oct 31, 2026 · @Turki

Counting rules

Every Person 2 method runs in O(1), O(k) or O(n); the costliest is `RideList.findByRiderName` at O(n·k). Each table gives steps per execution (s/e), frequency and total for the worst case.

- n = number of rides in `RideList`; k = number of riders on one shared ride.
- Each simple statement or condition counts as 1 step.
- A call to a constant-time method counts as 1 step: getters, `compareTo`, `equalsIgnoreCase`, and the `LinkedList` methods `empty`, `findFirst`, `findNext`, `retrieve`, `last` and `insert`.
- String concatenation counts as 1 step.
- `LinkedList.remove()` walks from the head to find the previous node, so it counts as k + 3 in the worst case.

RideList

All `RideList` methods are O(n) except `size`, and `findByRiderName` is O(n·k) because it also scans shared-ride participants.

addRide (worst case: new ride goes last, ID not a duplicate)

Statement	s/e	Freq	Total
`if (ride == null) return false`	1	1	1
`p = head`	1	1	1
`while (p != null)`	1	n+1	n+1
`if (p.ride.getRideId() == ride.getRideId())`	1	n	n
`p = p.next`	1	n	n

Statement	s/e	Freq	Total
`newNode = new RideNode(ride)`	1	1	1
`if` head is null or `ride.compareTo(head.ride) < 0`	1	1	1
`prev = head`	1	1	1
`while` `prev.next != null` and `compareTo(ride) <= 0`	1	n	n
`prev = prev.next`	1	n−1	n−1
`newNode.next = prev.next; prev.next = newNode`	2	1	2
`size++`	1	1	1
`return true`	1	1	1
Total			**5n + 9 → O(n)**

removeRideById (worst case: ride not in the list)

Statement	s/e	Freq	Total
`prev = null`	1	1	1
`p = head`	1	1	1
`while (p != null)`	1	n+1	n+1
`if (p.ride.getRideId() == rideId)`	1	n	n
`prev = p`	1	n	n
`p = p.next`	1	n	n
`return false`	1	1	1
Total			**4n + 4 → O(n)**

getAllAlphabetically

The list is already sorted, so one pass copies it. This meets the O(n) listing requirement.

Statement	s/e	Freq	Total
`result = new LinkedList`	1	1	1
`p = head`	1	1	1
`while (p != null)`	1	n+1	n+1
`result.insert(p.ride)`	1	n	n
`p = p.next`	1	n	n
`return result`	1	1	1
Total			**3n + 4 → O(n)**

findByPickupLocation (worst case: every ride matches)

Statement	s/e	Freq	Total
`result = new LinkedList`	1	1	1
`if (pickupLocation == null)`	1	1	1
`p = head`	1	1	1
`while (p != null)`	1	n+1	n+1
`if (...equalsIgnoreCase(pickupLocation))`	1	n	n
`result.insert(p.ride)`	1	n	n
`p = p.next`	1	n	n
`return result`	1	1	1
Total			**4n + 5 → O(n)**

hasRiderNamed (private helper; worst case: shared ride, k riders, no match)

Statement	s/e	Freq	Total
`if (ride instanceof IPrivateRide)`	1	1	1

Statement	s/e	Freq	Total
`participants = getParticipants()`	1	1	1
`if (participants.empty())`	1	1	1
`participants.findFirst()`	1	1	1
`if (retrieve().getName().equals(name))`	1	k	k
`if (participants.last())`	1	k	k
`participants.findNext()`	1	k−1	k−1
`return false`	1	1	1
Total			**3k + 4 → O(k)**

findByRiderName (worst case: every ride is shared with k riders)

Statement	s/e	Freq	Total
`result = new LinkedList`	1	1	1
`if (riderFullName == null)`	1	1	1
`p = head`	1	1	1
`while (p != null)`	1	n+1	n+1
`if (hasRiderNamed(...))`	1 + (3k+4)	n	n(3k+5)
`result.insert(p.ride)`	1	n	n
`p = p.next`	1	n	n
`return result`	1	1	1
Total			**3nk + 8n + 5 → O(n·k)**

A shared ride has only a few riders, so k is bounded by a small constant and this is effectively O(n).

SharedRide

Every `SharedRide` operation that touches participants is O(k), because the participant list is searched from its first node.

hasRider (worst case: rider not found)

Statement	s/e	Freq	Total
`if (participants.empty())`	1	1	1
`participants.findFirst()`	1	1	1
`while (true)`	1	k	k
`if (retrieve().getId() == riderId)`	1	k	k
`if (participants.last())`	1	k	k
`participants.findNext()`	1	k−1	k−1
`return false`	1	1	1
Total			**4k + 2 → O(k)**

addParticipant (worst case: new rider, k riders already on the ride)

Statement	s/e	Freq	Total
`if` rider is null or `hasRider(id)`	1 + (4k+2)	1	4k + 3
`if (!participants.empty())`	1	1	1
`participants.findFirst()`	1	1	1
`while (!participants.last())`	1	k	k
`participants.findNext()`	1	k−1	k−1
`participants.insert(rider)`	1	1	1
`return true`	1	1	1
Total			**6k + 6 → O(k)**

removeParticipantById (worst case: rider is the last one)

Statement	s/e	Freq	Total
`if (!hasRider(riderId))`	1 + (4k+1)	1	4k + 2
`participants.remove()`	k + 3	1	k + 3
`return true`	1	1	1
Total			**5k + 6 → O(k)**

toString (worst case: k riders)

Statement	s/e	Freq	Total
`riders = ""`	1	1	1
`if (!participants.empty())`	1	1	1
`participants.findFirst()`	1	1	1
`while (true)`	1	k	k
`r = participants.retrieve()`	1	k	k
`riders = riders + name + id`	1	k	k
`if (participants.last()) break`	1	k	k
`riders = riders + ", "`	1	k−1	k−1
`participants.findNext()`	1	k−1	k−1
`if (riders.equals(""))`	1	1	1
`return ...`	1	1	1
Total			**6k + 3 → O(k)**

Constant-time methods

The remaining methods run a fixed number of steps regardless of n or k, so each is O(1).

Class	Method	Steps (worst case)	Total	Big-O
`DateTime`	constructor (5 checks + `daysInMonth` ≤ 5 + 5 assignments)	5 + 5 + 5	15	O(1)
`DateTime`	`compareTo` (4 false `if`s + final `return`)	4 + 1	5	O(1)
`DateTime`	`getYear`, `getMonth`, `getDay`, `getHour`, `getMinute`	1 each	1	O(1)
`DateTime`	`format`, `toString`	1	1	O(1)
`Ride`	constructor (2 location checks, null check, `compareTo` check 1 + 5, 2 for the ID, 5 assignments)	1 + 1 + 1 + 6 + 2 + 5	16	O(1)
`Ride`	getters, `setDriver`, `getRideId`	1 each	1	O(1)
`Ride`	`setPickupLocation`, `setDropoffLocation` (check + assign)	1 + 1	2	O(1)
`Ride`	`compareTo`	1	1	O(1)
`Ride`	`toString` (`driverText = "none"`, `if`, assign, `return`)	4	4	O(1)
`PrivateRide`	constructor (`Ride` constructor 16 + null check 1 + assign 1)	16 + 1 + 1	18	O(1)
`PrivateRide`	`getRider`	1	1	O(1)
`PrivateRide`	`setRider` (check + assign)	1 + 1	2	O(1)
`PrivateRide`	`hasRider`, `toString`	1	1	O(1)
`SharedRide`	constructor (`Ride` constructor 16 + list creation 1)	16 + 1	17	O(1)
`SharedRide`	`getParticipants`, `isEmpty`	1	1	O(1)
`RideList`	`size`	1	1	O(1)

Final summary

The worst-case results for all Person 2 methods; `SharedRide.removeParticipantById` includes the cost of `LinkedList.remove()`, so cite the `LinkedList` author's analysis for that step.

Class	Method	Total (worst case)	Big-O
`DateTime`	all methods	constant (≤ 15)	O(1)
`Ride`	all methods	constant (≤ 16)	O(1)
`PrivateRide`	all methods	constant (≤ 18)	O(1)
`SharedRide`	`hasRider`	4k + 2	O(k)
`SharedRide`	`addParticipant`	6k + 6	O(k)
`SharedRide`	`removeParticipantById`	5k + 6	O(k)
`SharedRide`	`toString`	6k + 3	O(k)
`SharedRide`	other methods	constant	O(1)
`RideList`	`addRide`	5n + 9	O(n)
`RideList`	`removeRideById`	4n + 4	O(n)
`RideList`	`getAllAlphabetically`	3n + 4	O(n)
`RideList`	`findByPickupLocation`	4n + 5	O(n)
`RideList`	`findByRiderName`	3nk + 8n + 5	O(n·k)
`RideList`	`size`	1	O(1)

Time Complexity Analysis class RideSharingSystem

RideSharingSystem() (constructor)

code	frequency
riderList = new RiderList();	1
driverList = new DriverList();	1
rideList = new RideList();	1
Big O	O(1)

addDriver(IDriver driver)

code	frequency
if (driver == null) return false;	1
if (driverList.findById(driver.getId()) != null) return false;	n
if (driverList.findByVehiclePlate(driver.getVehiclePlate()) != null) return false;	n
return driverList.add(driver);	n
Big O	O(n)

searchRiderById(int riderId)

code	frequency
return riderList.findById(riderId);	n
Big O	O(n)

searchRiderByEmail(String email)

code	frequency
return riderList.findByEmail(email);	n
Big O	O(n)

searchRidersByName(String fullName)

code	frequency
return riderList.findByName(fullName);	n
Big O	O(n)

searchRidersByHomeCity(String homeCity)

code	frequency
return riderList.findByHomeCity(homeCity);	n
Big O	O(n)

getAllRiders()

code	frequency
return riderList.getAll();	n
Big O	O(n)

searchDriverById(int driverId)

code	frequency
return driverList.findById(driverId);	n
Big O	O(n)

searchDriverByVehiclePlate(String vehiclePlate)

code	frequency
return driverList.findByVehiclePlate(vehiclePlate);	n
Big O	O(n)

searchDriversByVehicleType(VehicleType vehicleType)

code	frequency
return driverList.findByVehicleType(vehicleType);	n
Big O	O(n)

getAllDrivers()

code	frequency
return driverList.getAll();	n
Big O	O(n)

removeRider(int riderId)

code	frequency
IRider rider = riderList.findById(riderId);	n
if (rider == null) return false;	1
LinkedList<IRide> history = rider.getRideHistory();	1
if (!history.empty())	1
history.findFirst();	1
while (true)	n
IRide ride = history.retrieve();	n
if (ride instanceof ISharedRide)	n
ISharedRide shared = (ISharedRide) ride;	n
shared.removeParticipantById(riderId);	n^2
if (shared.isEmpty())	n
rideList.removeRideById(shared.getRideId());	n^2
removeFromHistory(shared.getDriver().getRideHistory(), shared.getRideId());	n^2
rideList.removeRideById(ride.getRideId());	n^2
removeFromHistory(ride.getDriver().getRideHistory(), ride.getRideId());	n^2
if (history.last()) break;	n
history.findNext();	n
return riderList.removeById(riderId);	n
Big O	$O(n^2)$

removeFromHistory(LinkedList<IRide> history, int rideId)

code	frequency
if (history.empty()) return;	1
history.findFirst();	1
while (true)	n
if (history.retrieve().getRideId() == rideId)	n
history.remove();	n
return;	1
if (history.last()) return;	n
history.findNext();	n
Big O	O(n)

removeFromRidersHistories(IRide ride)

code	frequency
if (ride instanceof ISharedRide)	1
LinkedList<IRider> participants = ((ISharedRide) ride).getParticipants();	1
if (participants.empty()) return;	1
participants.findFirst();	1
while (true)	n
removeFromHistory(participants.retrieve().getRideHistory(), ride.getRideId());	n^2
if (participants.last()) break;	n
participants.findNext();	n
removeFromHistory(((IPrivateRide) ride).getRider().getRideHistory(), ride.getRideId());	n
Big O	$O(n^2)$

schedulePrivateRide(String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation, int riderId, int driverId)

code	frequency
if (pickupTime == null || dropoffTime == null) return false;	1
IRider rider = riderList.findById(riderId);	n
IDriver driver = driverList.findById(driverId);	n
if (rider == null || driver == null) return false;	1
if (hasConflict(rider, pickupTime, dropoffTime)) return false;	n
if (hasConflict(driver, pickupTime, dropoffTime)) return false;	n
IPrivateRide ride;	1
try { ride = new PrivateRide(pickupLocation, pickupTime, dropoffTime, dropoffLocation, driver, rider); }	1
catch (IllegalArgumentException e) { return false; }	1
if (!rideList.addRide(ride)) return false;	n
addToHistory(rider.getRideHistory(), ride);	1
addToHistory(driver.getRideHistory(), ride);	1
return true;	1
Big O	O(n)

scheduleSharedRide(String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation, int[] riderIds, int driverId)

code	frequency
if (pickupTime == null || dropoffTime == null) return false;	1
if (riderIds == null || riderIds.length == 0) return false;	1
IDriver driver = driverList.findById(driverId);	n
if (driver == null || hasConflict(driver, pickupTime, dropoffTime)) return false;	n
for (int i = 0; i < riderIds.length; i++)	n
IRider rider = riderList.findById(riderIds[i]);	n^2
if (rider == null || hasConflict(rider, pickupTime, dropoffTime)) return false;	n^2
ISharedRide ride;	1
try { ride = new SharedRide(pickupLocation, pickupTime, dropoffTime,dropoffLocation, driver);}	1
catch (IllegalArgumentException e) {return false; }	1
for (int i = 0; i < riderIds.length; i++)	n
IRider rider = riderList.findById(riderIds[i]);	n^2
if (ride.addParticipant(rider))	n^2
addToHistory(rider.getRideHistory(), ride);	n
if (!rideList.addRide(ride)) return false;	n
addToHistory(driver.getRideHistory(), ride);	1
return true;	1
Big O	$O(n^2)$

searchRidesByPickupLocation(String pickupLocation)

code	frequency
return rideList.findByPickupLocation(pickupLocation);	n
Big O	O(n)

searchRidesByRiderName(String riderName)

code	frequency
return rideList.findByRiderName(riderName);	n^2
Big O	$O(n^2)$

getSharedRideParticipants(String pickupLocation)

code	frequency
LinkedList<IRider> result = new LinkedList<IRider>();	1
LinkedList<IRide> rides = rideList.findByPickupLocation(pickupLocation);	n
if (rides.empty()) return result;	1
rides.findFirst();	1
while (true)	n
IRide ride = rides.retrieve();	n
if (ride instanceof ISharedRide)	n
LinkedList<IRider> participants = ((ISharedRide) ride).getParticipants();	n
if (!participants.empty())	n
participants.findFirst();	n
while (true)	n^2
result.insert(participants.retrieve());	n^2
if (participants.last()) break;	n^2
participants.findNext();	n^2
if (rides.last()) break;	n
rides.findNext();	n
return result;	1
Big O	$O(n^2)$

getAllRidesAlphabetically()

code	frequency
return rideList.getAllAlphabetically();	n
Big O	O(n)

loadRidersFromCSV(String ridersFilePath)

code	frequency
try (BufferedReader br = new BufferedReader(new FileReader(ridersFilePath)))	1
String line;	1
while ((line = br.readLine()) != null)	n
String[] v = line.split(",");	n
IRider rider = new Rider(Integer.parseInt(v[0].trim()), v[1].trim(), v[3].trim(), v[2].trim(), v[4].trim());	n
addRider(rider);	n^2
catch (IllegalArgumentException | ArrayIndexOutOfBoundsException e) { continue; }	n
return true;	1
catch (IOException e) { return false; }	1
Big O	$O(n^2)$

loadDriversFromCSV(String driversFilePath)

code	frequency
try (BufferedReader br = new BufferedReader(new FileReader(driversFilePath)))	1
String line;	1
while ((line = br.readLine()) != null)	n
String[] v = line.split(",");	n
VehicleType type = VehicleType.valueOf(v[4].trim());	n
IDriver driver = new Driver(Integer.parseInt(v[0].trim()), v[1].trim(), v[2].trim(), v[3].trim(), type);	n
addDriver(driver);	n^2
catch (IllegalArgumentException | ArrayIndexOutOfBoundsException e) { continue; }	n
return true;	1
catch (IOException e) { return false; }	1
Big O	$O(n^2)$

loadRidesFromCSV(String ridesFilePath)

code	frequency
try (BufferedReader br = new BufferedReader(new FileReader(ridesFilePath)))	1
String line;	1
while ((line = br.readLine()) != null)	n
String[] v = line.split(",");	n
String type = v[0].trim();	n
IDateTime pickup = parseDateTime(v[2]);	n
IDateTime dropoff = parseDateTime(v[3]);	n
int driverId = Integer.parseInt(v[5].trim());	n
String[] idTexts = v[6].trim().split(";");	n
int[] riderIds = new int[idTexts.length];	n
for (int i = 0; i < idTexts.length; i++)	n^2
riderIds[i] = Integer.parseInt(idTexts[i].trim());	n^2
if (type.equals("PRIVATE"))	n
schedulePrivateRide(v[1].trim(), pickup, dropoff, v[4].trim(), riderIds[0], driverId);	n^2
else if (type.equals("SHARED"))	n
scheduleSharedRide(v[1].trim(), pickup, dropoff, v[4].trim(), riderIds, driverId);	n^3
catch (IllegalArgumentException | ArrayIndexOutOfBoundsException e) { continue; }	n
return true;	1
catch (IOException e) { return false; }	1
Big O	$O(n^3)$

addRider(IRider rider)

code	frequency
if (rider == null) return false;	1
if (riderList.findById(rider.getId()) != null) return false;	n
return riderList.add(rider);	n
Big O	O(n)

removeDriver(int driverId)

code	frequency
IDriver driver = driverList.findById(driverId);	n
if (driver == null) return false;	1
LinkedList<IRide> history = driver.getRideHistory();	1
if (!history.empty())	1
history.findFirst();	1
while (true)	n
IRide ride = history.retrieve();	n
rideList.removeRideById(ride.getRideId());	n^2
removeFromRidersHistories(ride);	n^3
if (history.last()) break;	n
history.findNext();	n
return driverList.removeById(driverId);	n
Big O	$O(n^3)$

parseDateTime(String text)

code	frequency
String[] parts = text.trim().split(" ");	1
String[] date = parts[0].split("/");	1
String[] time = parts[1].split(":");	1
return new DateTime(Integer.parseInt(date[2]), Integer.parseInt(date[0]), Integer.parseInt(date[1]), Integer.parseInt(time[0]), Integer.parseInt(time[1]));	1
Big O	O(1)

hasConflict(IPerson person, IDateTime start, IDateTime end)

code	frequency
LinkedList<IRide> history = person.getRideHistory();	1
if (history.empty()) return false;	1
history.findFirst();	1
while (true)	n
IRide existing = history.retrieve();	n
if (start.compareTo(existing.getDropoffTime()) < 0 && existing.getPickupTime().compareTo(end) < 0) return true;	n
if (history.last()) break;	n
history.findNext();	n
return false;	1
Big O	O(n)

addToHistory(LinkedList<IRide> history, IRide ride)

code	frequency
if (!history.empty()) history.findFirst();	1
history.insert(ride);	1
Big O	O(1)

Time Complexity Analysis class Main

main(String[] args)

code	frequency
RideSharingSystem system = new RideSharingSystem();	1
if (!system.loadRidersFromCSV("riders_100.csv"))	n^2
if (!system.loadDriversFromCSV("drivers_30.csv"))	n^2
if (!system.loadRidesFromCSV("rides_40.csv"))	n^3
int choice = -1;	1
while (choice != 0) (one iteration)	1
printMenu();	1
choice = readInt("Enter your choice: ");	1
switch (choice)	1
case 1: printList(system.getAllRiders());	n
case 2: printList(system.searchRidersByHomeCity(readLine("Home city: ")));	n
case 3: printList(system.searchRidersByName(readLine("Full name: ")));	n
case 4: addRiderOption(system);	n
case 5: printList(system.getAllDrivers());	n
case 6: printList(system.searchDriversByVehicleType(readVehicleType()));	n
case 7: addDriverOption(system);	n
case 8: printList(system.getAllRidesAlphabetically());	n
case 9: printList(system.searchRidesByPickupLocation(readLine("Pickup location: ")));	n
case 10: printList(system.searchRidesByRiderName(readLine("Rider name: ")));	n^2
case 11: printList(system.getSharedRideParticipants(readLine("Pickup location: ")));	n^2
case 12: addRideOption(system);	n^2
case 0: System.out.println("Goodbye.");	1
default: System.out.println("Invalid choice. Please try again.");	1
Big O	$O(n^3)$

printMenu()

code	frequency
System.out.println(...); // 14 lines: a blank line, the 12 options and Exit	14
Big O	O(1)

printList(LinkedList<T> list)

code	frequency
if (list == null || list.empty())	1
System.out.println("No results found.");	1
return;	1
list.findFirst();	1
while (true)	n
System.out.println(list.retrieve());	n
if (list.last()) break;	n
list.findNext();	n
Big O	O(n)

Note: Here n is the number of elements in the list that is printed.

readLine(String prompt)

code	frequency
System.out.print(prompt);	1
String line = input.readLine();	1
return line == null ? "" : line.trim();	1
Big O	O(1)

readInt(String prompt)

code	frequency
while (true)	1
String text = readLine(prompt);	1
return Integer.parseInt(text);	1
catch (NumberFormatException e) { System.out.println("Please enter a valid number."); }	1
Big O	O(1)

Note: The frequencies are for one attempt. The loop repeats only while the user types something that is not a number, so it does not depend on n.

readVehicleType()

code	frequency
while (true)	1
String text = readLine("Vehicle type (SEDAN, LUXURY_SEDAN, SUV, VAN): ");	1
return VehicleType.valueOf(text.toUpperCase());	1
catch (IllegalArgumentException e) { System.out.println("Invalid vehicle type."); }	1
Big O	O(1)

Note: The frequencies are for one attempt. The enum has only four values, so the lookup does not depend on n.

addRiderOption(RideSharingSystem system)

code	frequency
int id = readInt("Rider ID: ");	1
String name = readLine("Full name: ");	1
String phone = readLine("Phone number (10 digits): ");	1
String email = readLine("Email: ");	1
String city = readLine("Home city: ");	1
IRider rider = new Rider(id, name, phone, email, city);	1
if (system.addRider(rider))	n
System.out.println("Rider added successfully.");	1
System.out.println("Failed to add rider: the ID already exists.");	1
catch (IllegalArgumentException e) { System.out.println("Failed to add rider: " + e.getMessage()); }	1
Big O	O(n)

addDriverOption(RideSharingSystem system)

code	frequency
int id = readInt("Driver ID: ");	1
String name = readLine("Full name: ");	1
String phone = readLine("Phone number (10 digits): ");	1
String plate = readLine("Vehicle plate (e.g. ABC1234): ");	1
VehicleType type = readVehicleType();	1
IDriver driver = new Driver(id, name, phone, plate, type);	1
if (system.addDriver(driver))	n
System.out.println("Driver added successfully.");	1
System.out.println("Failed to add driver: the ID or the plate already exists.");	1
catch (IllegalArgumentException e) { System.out.println("Failed to add driver: " + e.getMessage()); }	1
Big O	O(n)

addRideOption(RideSharingSystem system)

code	frequency
String type = readLine("Ride type (private/shared): ").toLowerCase();	1
if (!type.equals("private") && !type.equals("shared"))	1
System.out.println("Invalid ride type."); return;	1
String pickupLocation = readLine("Pickup location: ");	1
IDateTime pickupTime = readDateTime("Pickup time (MM/DD/YYYY HH:MM): ");	1
IDateTime dropoffTime = readDateTime("Drop-off time (MM/DD/YYYY HH:MM): ");	1
String dropoffLocation = readLine("Drop-off location: ");	1
int driverId = readInt("Driver ID: ");	1
boolean scheduled;	1
if (type.equals("private"))	1
int riderId = readInt("Rider ID: ");	1
scheduled = system.schedulePrivateRide(pickupLocation, pickupTime, dropoffTime, dropoffLocation, riderId, driverId);	n
int[] riderIds = readRiderIds("Rider IDs separated by commas: ");	n

if (riderIds.length < 2) { System.out.println("A shared ride needs at least 2 riders."); return; }	1
scheduled = system.scheduleSharedRide(pickupLocation, pickupTime, dropoffTime, dropoffLocation, riderIds, driverId);	n^2
if (scheduled)	1
System.out.println("Ride scheduled successfully.");	1
System.out.println("Failed to schedule the ride.");	1
Big O	$O(n^2)$

Note: The worst case is the shared ride, because scheduleSharedRide is $O(n^2)$. A private ride costs O(n).

readDateTime(String prompt)

code	frequency
while (true)	1
String text = readLine(prompt);	1
String[] parts = text.split(" ");	1
String[] date = parts[0].split("/");	1
String[] time = parts[1].split(":");	1
return new DateTime(Integer.parseInt(date[2]), Integer.parseInt(date[0]), Integer.parseInt(date[1]), Integer.parseInt(time[0]), Integer.parseInt(time[1]));	1
catch (IllegalArgumentException | ArrayIndexOutOfBoundsException e) { System.out.println("Invalid date..."); }	1
Big O	O(1)

Note: The frequencies are for one attempt. The loop repeats only while the user types an invalid date.

readRiderIds(String prompt)

code	frequency
while (true)	1
String[] parts = readLine(prompt).split(",");	n
int[] ids = new int[parts.length];	n
for (int i = 0; i < parts.length; i++)	n
ids[i] = Integer.parseInt(parts[i].trim());	n
return ids;	1
catch (NumberFormatException e) { System.out.println("Please enter numbers separated by commas."); }	1
Big O	O(n)